

Federated Learning

March 3, 2026

1 Centralized Training Workflow

Traditional machine learning systems follow a centralized approach where data from many sources is gathered in one place to train a model [1]. As shown in Figure 1, data is first collected from various devices and locations, then sent to a central server, which is usually hosted on a cloud platform. The model is trained on this combined dataset, and once ready, it is sent back to the end devices for use [1].

In intelligent transportation systems, for example, buses, roadside sensors, and other infrastructure devices constantly produce data. This data is uploaded to cloud servers where it is processed and used for training [2]. The goal of centralized learning is to find a model that reduces the overall error across all the collected training samples [3].

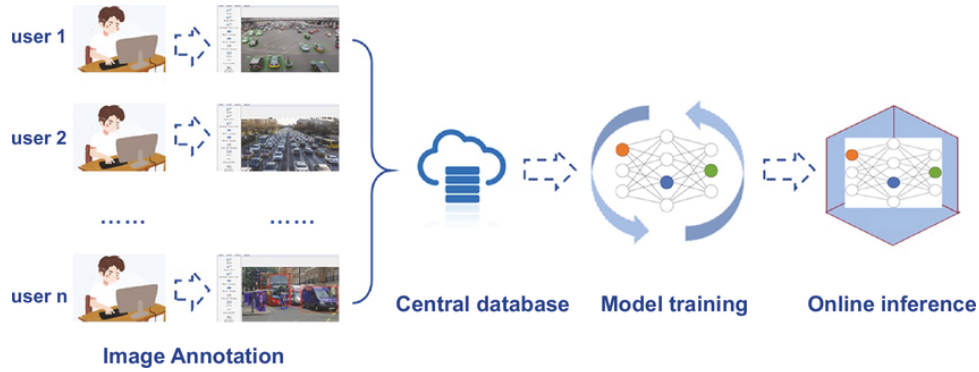


Figure 1: Centralized Training Workflow [4].

2 Limits of Centralized Machine Learning

Centralized machine learning faces several important limitations that make it poorly suited for modern transportation applications.

2.1 Privacy Vulnerabilities

Storing all data in one central location makes it an attractive target for attackers [5]. If that central storage is breached, personal or sensitive records can be exposed, which damages public trust in machine learning systems [5].

Beyond the risk of storing data in one place, the process of uploading data itself creates a vulnerability. When data travels from a bus or a vehicle to a remote cloud server, it passes through networks that can be intercepted. During this transit phase, attackers may capture sensitive information before it even reaches the central server. This means that data is at risk not only when it is stored but also while it is being transmitted [5].

In the context of bus fleets, for instance, onboard sensors collect passenger counts and route information. Uploading all this to a single cloud server means that a single breach could expose the travel patterns and habits of thousands of passengers at once.

2.2 Single Point of Failure

The central server is a single point of failure. If it goes down or is compromised, the entire system stops working, leading to service interruptions and loss of trust [2]. For a bus fleet management system, this would mean that all predictive maintenance, route optimization, and scheduling services could fail simultaneously [12].

2.3 Communication Costs

Sending large amounts of raw data to a central server comes with significant costs. These costs can be divided into two main categories [1].

First, there are **transport costs**, which refer to the bandwidth and network resources needed to move data from each device to the central server. When thousands of buses send sensor readings, video feeds, or diagnostic logs every day, the amount of data transferred becomes very large, and the cost of that data transfer grows accordingly [1].

Second, there are **storage costs**, which refer to the expense of keeping all that data on the central server. As the fleet grows and data accumulates over time, the storage infrastructure must be expanded continuously, adding to operational expenses [1].

2.4 Scalability Constraints

As data volume grows, centralized systems struggle to keep up. Adding more powerful hardware can help up to a point, but there are natural limits to how much a single system can handle [5]. When thousands of buses generate data at the same time, the processing load on one central entity becomes very heavy and difficult to manage [13].

2.5 Regulatory Compliance

In several countries and regions, regulations such as the General Data Protection Regulation (GDPR) in Europe impose strict rules on how personal data is collected, stored, and processed. These regulations make centralized data collection legally difficult because gathering all user data in one location increases the risk of non-compliance [6]. Any organization handling private data must take great care to limit privacy risks, and centralized approaches make this harder to achieve [1].

2.6 The IID Assumption Problem

Centralized machine learning typically assumes that the training data is Independent and Identically Distributed (IID). This means that every data sample is assumed to come from the same underlying distribution, and each sample is independent from the others. In practice, this assumption rarely holds when data comes from different sources [3].

For example, buses operating in a city center will produce data that looks very different from buses running on suburban or rural routes. The driving patterns, passenger loads, road conditions, and even weather exposure vary greatly depending on the location and time of day. When all this data is mixed together and treated as if it comes from one uniform source, the resulting model may perform poorly on specific subgroups of the data [1]. This mismatch between the IID assumption and the reality of heterogeneous data is a fundamental limitation of centralized approaches.

3 Motivation for Decentralized Approach

The problems described above motivate the shift toward decentralized learning. In a decentralized approach, instead of gathering all data in one central server, the training process is spread across multiple devices or nodes. Each device works with its own local data, and only the results of the training (such as model updates) are shared, not the raw data itself.

There are two main types of decentralized approaches: **Distributed Learning** and **Federated Learning**. Although both involve training across multiple devices, they differ in their goals, architecture, and constraints.

3.1 Distributed Learning

Distributed learning focuses on making the training process faster by splitting the work across multiple machines, usually within a controlled environment such as a data center. The data is typically divided evenly among the machines, and they communicate through fast, reliable networks. All machines tend to have similar hardware capabilities. The main goal of distributed learning is to speed up training and handle larger datasets, not to protect privacy [8, 2].

As shown in Figure 2, the full dataset is divided into equal parts and sent to different worker machines. Each worker trains on its portion and sends the computed gradients back to a coordinator, which combines them to update the global model. The communication happens over high-speed networks, and the data is typically assumed to be IID across workers [3].

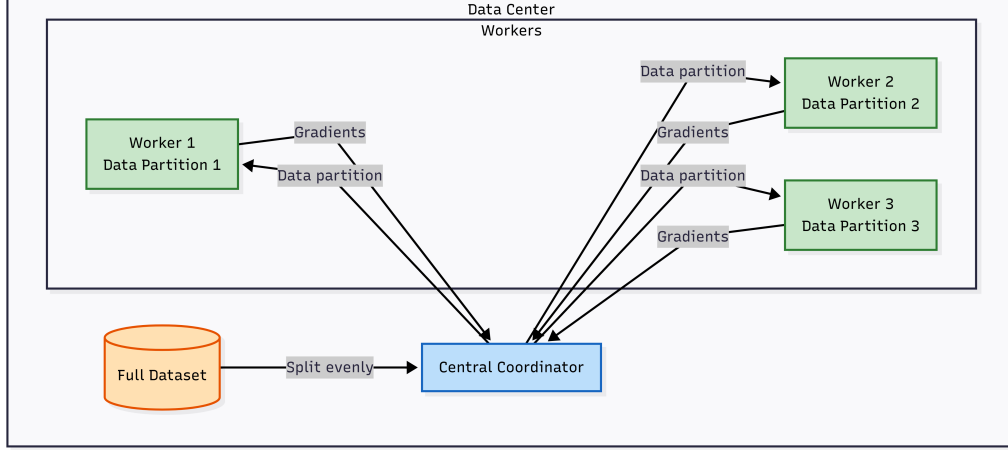


Figure 2: Distributed Learning Architecture (based on concepts described in [8]).

However, distributed learning has its own drawbacks. Since data must still be copied and sent to the worker machines, privacy is not protected. The approach also requires a controlled network environment with fast and reliable connections, which is not always possible in real-world scenarios such as transportation systems. Furthermore, it does not handle situations where data is naturally non-IID or where devices have very different hardware capabilities [8].

3.2 Federated Learning

Federated learning takes a different approach. Instead of moving data to the machines, the model is sent to where the data already lives. Each device trains the model locally on its own data and sends only the model updates back to a central server. The raw data never leaves the device [3].

As shown in Figure 3, in a federated learning setup applied to a bus fleet, each bus holds its own sensor data locally. The central server sends the current global model to the buses. Each bus trains the model on its local data and sends the updated model weights back to the server, which combines all the updates to improve the global model. The key advantage is that sensitive data such as passenger information and GPS traces stays on the bus at all times [3, 1].

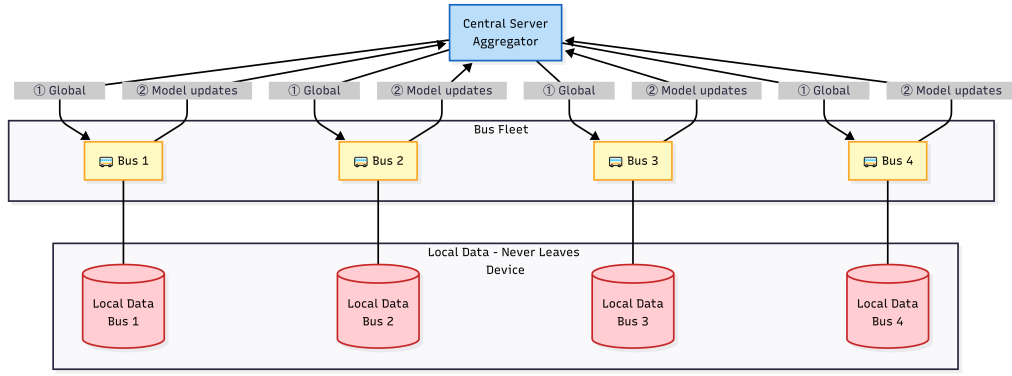


Figure 3: Federated Learning Architecture [3].

3.3 Advantages of Federated Learning

This approach brings several important benefits for transportation systems.

Privacy preservation. Since data stays on the local device, sensitive information never travels across the network. For bus fleets, this means passenger data, route details, and operational logs remain protected without needing to be uploaded to a cloud server [7, 1].

Reduced communication overhead. Instead of sending large volumes of raw sensor data, only compact model updates are transmitted. This significantly reduces the bandwidth needed, which is especially important for buses that may rely on mobile or intermittent network connections [8, 1].

Leveraging edge computing. Modern onboard computing units in buses and vehicles have enough processing power to train models locally without depending on cloud resources [3, 1].

Distributed knowledge aggregation. Each bus operates under different conditions depending on its route, weather, traffic, and passenger load. Federated learning allows the global model to benefit from all this diversity without centralizing the data, leading to better generalization across the fleet [6, 14, 5].

4 Comparison of Learning Approaches

Table 1 summarizes the key differences between centralized, distributed, and federated learning.

Table 1: Comparison of Centralized, Distributed, and Federated Learning

Criteria	Centralized	Distributed	Federated
Data location	Central server	Split across workers	Stays locally
Privacy	Not protected	Not protected	Protected by design
Communication	High (raw data transfer)	High (fast internal network)	Low (only model updates)
Data distribution	Assumed IID	Typically IID	Non-IID, unbalanced
Network requirements	Reliable internet	Reliable internet	Can work with slow or discontinuous connections
Scalability	Limited by server capacity	Good within data center	Scales to millions of devices
Main goal	Model accuracy	Training speed	Privacy-preserving collaboration
Device heterogeneity	Not applicable	Homogeneous	Heterogeneous
Regulatory compliance	Difficult	Difficult	Easier

As shown in Table 1, federated learning provides a strong combination of privacy protection, scalability, and reduced communication compared to centralized and standard distributed approaches [6, 3].

As illustrated in Figure 4, the evolution of machine learning has followed a clear path. Centralized learning came first, but its scalability limits led to the development of distributed learning. However, distributed learning still does not address privacy concerns, which motivated the creation of federated learning [15].

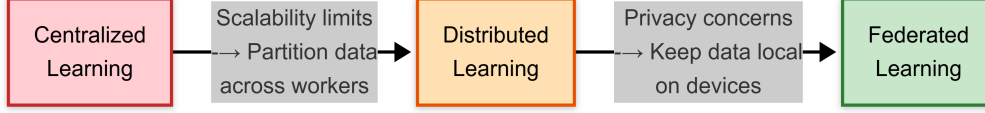


Figure 4: The evolution of machine learning paradigms [15].

5 Federated Learning: Concepts, Architecture, and Algorithms

5.1 Definition and Key Principles

Federated Learning is a machine learning approach where multiple clients work together to train a shared model, coordinated by a central server, while each client keeps its own data locally [3]. A global model is maintained by the server and improved over time through contributions from all participating devices [1].

The approach is built on several core principles [8]. First, raw data never leaves the client device, which is known as **data locality**. Second, only model updates are sent between devices and the server, making communication **model-centric**. Third, all clients work together to optimize a shared objective through **collaborative learning**. Fourth, the architecture is designed from the start to limit data exposure, following a **privacy-by-design** philosophy. Fifth, the system can accommodate clients with different hardware capabilities and different data distributions, which is referred to as **heterogeneity tolerance**.

5.2 Types of Federated Learning

Federated learning can be categorized in two ways: by how data is distributed across clients, and by how the system is organized architecturally. Understanding these two dimensions is important for selecting the right FL setup for a given application such as predictive maintenance in bus fleets.

5.2.1 By Data Distribution

Federated learning can be categorized into three groups according to how the *samples* (data records) and the *features* (data attributes) are distributed across participants: **horizontal federated learning**, **vertical federated learning**, and **federated transfer learning** [24].

As shown in Figure 5:

(a) Horizontal FL (HFL): parties collect data in the same format (same features), but each one has data from different users or events. It is the most common setting when many devices train the same model on their own local data without sharing raw records [24]. In a bus fleet context, this is the standard scenario: every bus collects the same types of sensor readings (engine temperature, GPS speed, fuel consumption), but from its own trips and passengers. Each bus trains locally and shares only model updates, which is exactly the HFL setup.

(b) Vertical FL (VFL): parties share the same users or events but each owns different features about them. Combining these complementary features requires privacy-preserving protocols such as entity alignment [24]. In transportation, VFL could apply when a bus operator and a road condition monitoring agency want to jointly train a maintenance model: the operator holds mechanical sensor data while the agency holds road surface quality data for the same routes, but neither wants to share its raw data directly.

(c) Federated Transfer Learning (FTL): parties have little overlap in both samples and features, so neither HFL nor VFL applies. Transfer learning is used to share knowledge across different domains [24]. In bus fleet maintenance, FTL could be useful when a transit authority wants to transfer knowledge

from a well-monitored fleet in one city to a new fleet in a different city where bus models and road conditions differ significantly.

For most bus predictive maintenance deployments, **Horizontal FL is the most appropriate choice**, as all buses share the same sensor types and the goal is to collaboratively learn a fault detection model from each bus’s private trip data.

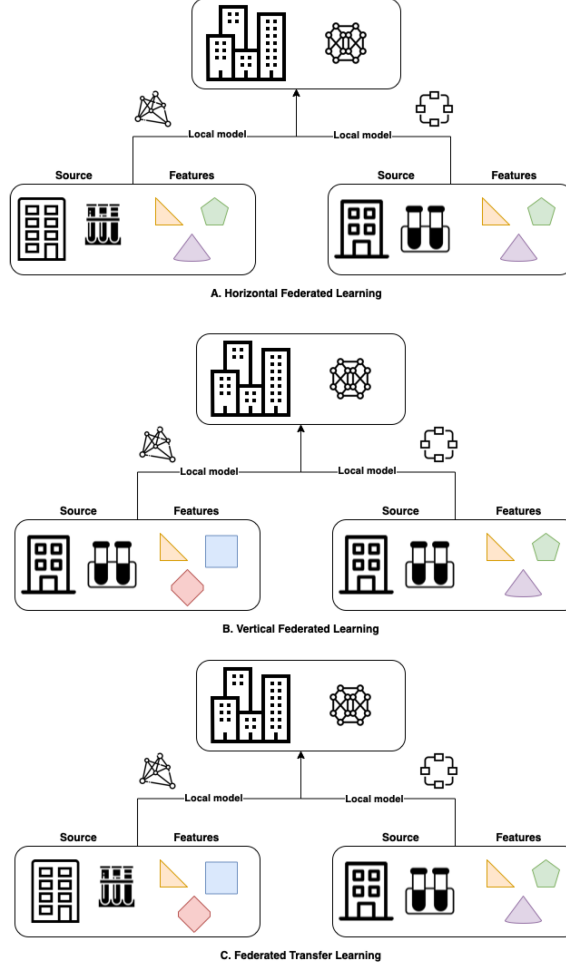


Figure 5: Data partition in (a) horizontal federated learning, (b) vertical federated learning, and (c) federated transfer learning. Source: [24].

5.2.2 By System Architecture

Federated learning can also be classified by **system architecture**, that is, how coordination and aggregation are organized between participants. In intelligent transportation systems (ITS), this choice directly affects scalability, communication latency, and robustness under intermittent connectivity. As shown in Figure 6, three common architectures are **centralized**, **hierarchical**, and **decentralized** FL [27, 28, 26].

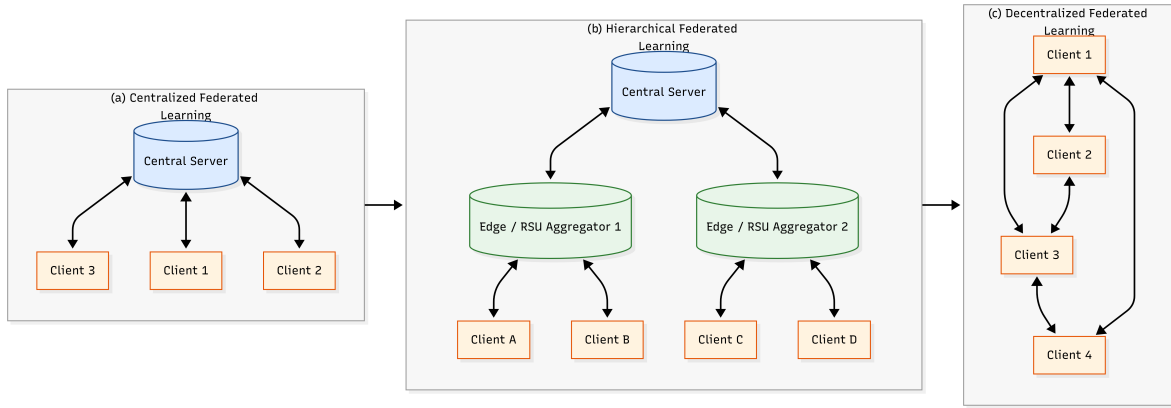


Figure 6: Comparison of federated learning system architectures: (a) centralized FL with a single coordinating server, (b) hierarchical FL with edge/RSU aggregators, (c) decentralized peer-to-peer FL.

Centralized FL (Figure 6a) uses a single server to broadcast the global model and aggregate all client updates. It is simple to implement and works well for small-to-medium fleets, but may become a bottleneck or a single point of failure at large scale [27, 28]. For a bus fleet with reliable internet access, this is a valid starting architecture.

Hierarchical FL (Figure 6b) introduces intermediate aggregators, such as roadside units (RSUs) or depot servers, that collect and aggregate updates from nearby buses before forwarding the combined result to the central cloud server. This reduces the volume of data transmitted to the cloud, lowers latency, and improves resilience when some buses have poor connectivity [28, 26]. For large urban bus networks with existing RSU infrastructure, hierarchical FL is the recommended architecture: it naturally maps to the geographic structure of bus routes and reduces the communication burden on individual vehicles.

Decentralized FL (Figure 6c) removes the central server entirely and allows buses to exchange model updates directly with neighbors over a peer-to-peer communication graph. This maximizes robustness and eliminates the single point of failure, but requires more complex coordination and depends on inter-vehicle connectivity [25, 29]. It is best suited for scenarios where vehicles are frequently co-located, such as at depots or along shared corridors.

In summary, the right architecture depends on fleet scale and infrastructure. For a small or medium operator, centralized FL offers simplicity. For a large city-scale deployment with RSU coverage, hierarchical FL provides the best balance of efficiency and resilience. Decentralized FL remains a promising direction for future vehicle-to-vehicle communication scenarios.

5.3 General Architecture of a Federated System

A federated learning system is made up of several components that work together [9].

The **central server**, also called the aggregator, is responsible for coordinating the entire learning process. It maintains the global model, collects updates from clients, aggregates them, and manages which clients participate in each training round.

The **clients**, or participants, are the devices that hold local private data. Each client trains the model on its own data and computes updates that are sent back to the server. Clients can be very different from each other in terms of computing power, network speed, and the amount of data they hold.

The **communication infrastructure** connects the server and the clients. It must be able to handle situations where connections are slow or interrupted, which is common in transportation scenarios. In some setups, edge nodes or roadside units act as intermediate aggregators to improve efficiency.

Finally, a **security and privacy layer** adds extra protection. This can include techniques such as secure aggregation, where the server can only see the combined result of all updates, or differential privacy, which adds noise to updates to prevent information leakage.

Figure 7 illustrates the complete architecture of a federated learning system with all its components.

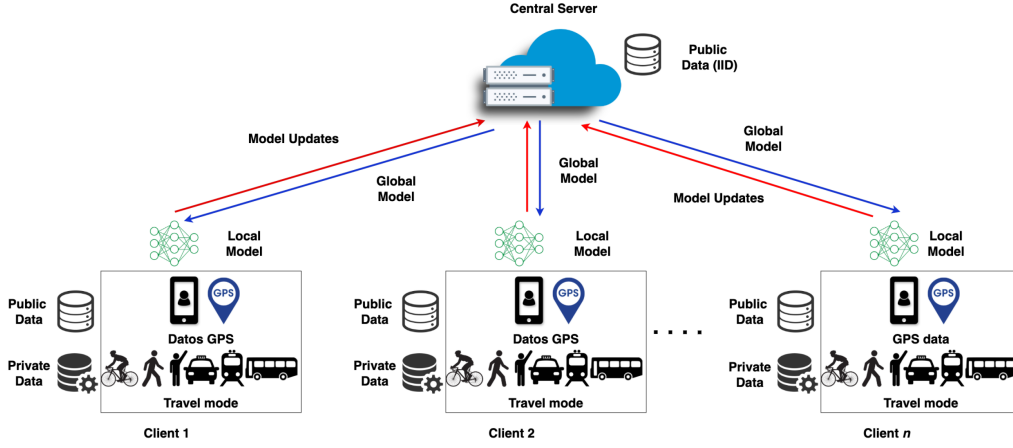


Figure 7: General architecture of a federated learning system. Source: [30].

5.4 Federated Learning Lifecycle

The federated learning process is iterative and works in repeated training rounds. In general, there are two main modes of operation depending on how the server coordinates with clients [3].

In **synchronous FL**, the server waits for all selected clients to finish their local training and send back their updates before aggregating and moving to the next round. This approach is straightforward and ensures that the global model is always updated with contributions from all selected participants. However, if even one client is slow or disconnects, the entire round is delayed.

In **asynchronous FL**, the server does not wait for all clients. Instead, it processes updates as they arrive, allowing training to continue even when some participants are temporarily unavailable. This mode is better suited for real-world deployments like bus fleets, where vehicles may lose connectivity in tunnels, rural areas, or during stops [6].

Regardless of the mode, each round follows the same general steps. The server first selects a group of available clients, then sends them the current version of the global model. Each client trains the model on its own local data for a number of steps and sends the updated parameters back to the server. The server combines all received updates, typically using a weighted average, to produce an improved global model. This cycle repeats until performance reaches a satisfactory level [3].

In practical terms, a base model is created by the server and pushed to participating devices. Each device refines the model using an optimization process such as stochastic gradient descent, then returns the updated weights. The server aggregates all weight updates and uses them to improve the global model [1].

Figure 8 illustrates the complete lifecycle of one federated learning round.

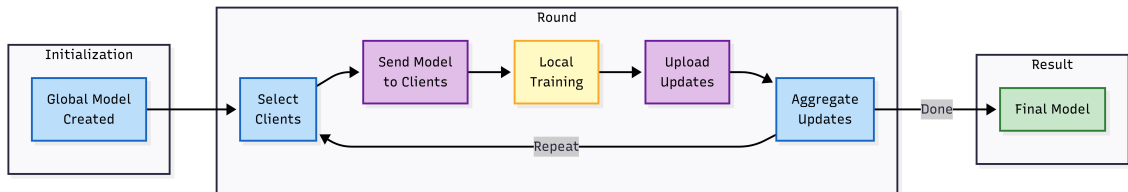


Figure 8: Federated Learning Lifecycle. Source: [3].

5.5 Core Algorithms

Before presenting the algorithms, it is useful to clarify the distinction between a **model** and an **algorithm** in the federated learning context. The *model* refers to the neural network or machine learning architecture that is trained on each device (for example, an LSTM or a feedforward network). The *algorithm* refers to the coordination strategy: how the model is distributed to clients, how local updates are computed and shared, and how the server aggregates contributions from multiple participants. All algorithms below are independent of the model choice; they define the training and aggregation procedure, not the model structure itself.

5.5.1 Federated Averaging (FedAvg)

FedAvg is the foundational federated learning algorithm, introduced by McMahan et al. [3]. The server initializes a global model with parameters w_0 and, in each round t , selects a random subset S_t of clients. It broadcasts the current global model w_t to all selected clients.

Each selected client k receives w_t and runs E local epochs of SGD on its private dataset $\mathcal{D}_k$ of size n_k , minimizing its local objective:

$$F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{D}_k} \ell(w; x_i, y_i) \quad (1)$$

After local training, each client sends back its updated parameters w_t^k . The server aggregates them using a weighted average:

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{n} w_t^k, \quad n = \sum_{k \in S_t} n_k \quad (2)$$

The global objective being optimized is:

$$\min_w F(w) = \sum_{k=1}^N \frac{n_k}{n} F_k(w) \quad (3)$$

FedAvg reduces communication significantly by allowing multiple local steps between aggregations and performs well even under non-IID data [3]. However, strong client heterogeneity may slow convergence, and theoretical guarantees are limited for non-convex objectives [20].

5.5.2 FedSGD

FedSGD is a special case of FedAvg where $E = 1$: each client computes a single gradient step before sending the update. The client update is:

$$w_t^k = w_t - \eta \nabla F_k(w_t) \quad (4)$$

and the server performs:

$$w_{t+1} = w_t - \eta \sum_{k \in S_t} \frac{n_k}{n} \nabla F_k(w_t) \quad (5)$$

This is equivalent to distributed gradient descent. It requires more communication rounds to converge but is the simplest possible federated algorithm [3, 1].

5.5.3 FedProx

FedProx [20] extends FedAvg to better handle heterogeneity by adding a proximal regularization term to each client's local objective:

$$h_k(w; w_t) = F_k(w) + \frac{\mu}{2} \|w - w_t\|^2 \quad (6)$$

where $\mu > 0$ controls how tightly the local model stays close to the global model. This prevents any single client from drifting too far, improving stability when clients have very different data or computing capabilities. When $\mu = 0$, FedProx reduces to standard FedAvg.

5.5.4 SCAFFOLD

SCAFFOLD [22] addresses *client drift*, which occurs when clients with non-IID data pull the global model in conflicting directions during local training. It introduces control variates c_k per client and a global control variate c to correct local gradient directions:

$$w_t^k \leftarrow w_t^k - \eta (\nabla F_k(w_t^k) - c_k + c) \quad (7)$$

After each round, the control variates are updated to reflect the gap between local and global gradient directions. SCAFFOLD achieves better convergence guarantees than FedAvg under non-IID data and is especially relevant for bus fleets where each vehicle has unique driving patterns.

5.5.5 FedNova

FedNova [23] addresses an inconsistency that arises when different clients perform different numbers of local steps τ_k . Directly averaging such updates introduces a bias in the global model. FedNova normalizes each client’s update before aggregation:

$$\Delta w_{\text{norm}}^k = \frac{1}{\tau_k} \Delta w^k, \quad \Delta w^k = w_t - w_t^k \quad (8)$$

ensuring that each contribution reflects the direction of improvement rather than the magnitude of local computation, which is important in heterogeneous environments with different device speeds.

5.5.6 DAAFL (Disparity-Aware Asynchronous FL)

DAAFL [6] is designed specifically for transportation fleets where clients have very different amounts of data and may disconnect during training. It dynamically adjusts each client’s aggregation weight based on both dataset size and update recency. Let α_k^t denote the weight of client k at round t :

$$\alpha_k^t \propto n_k \cdot f(\text{recency}_k^t) \quad (9)$$

where f is a recency function that gives more weight to clients underrepresented in recent rounds. This ensures that buses with smaller datasets or those that have dropped out temporarily still meaningfully influence the global model.

5.5.7 FedSA (Federated Semi-Asynchronous)

FedSA [21] handles clients with varying staleness, that is, clients that train on an outdated version of the global model. It begins with a synchronous phase and then transitions to asynchronous operation. The number of local epochs E_k for each client is reduced as its staleness s_k increases:

$$E_k = \max(1, E_{\max} - \lambda \cdot s_k) \quad (10)$$

where λ is a decay parameter and $E_{\max}$ is the maximum allowed local epochs. This prevents highly stale clients from submitting updates that diverge too far from the current global state.

5.6 Privacy and Security Mechanisms

Even though raw data stays on the device, the model updates themselves can still reveal information about the underlying data. Research has shown that it is possible to reconstruct training data from shared gradients [16]. Several defense mechanisms have been developed to address this.

Differential Privacy (DP) adds controlled noise to model updates before they are sent to the server, making it much harder for an attacker to reconstruct specific training examples. The trade-off is that too much noise can slow learning or reduce accuracy [28, 17].

Secure Multi-Party Computation (SMPC) uses cryptographic techniques to allow the server to aggregate client updates without seeing individual parameters in plain text. The server computes the weighted average without knowing any single client’s contribution [28, 18].

Homomorphic Encryption allows computations to be performed directly on encrypted data. Clients encrypt their model updates before sending them, and the server aggregates the encrypted values without decrypting them first [19].

Blockchain Integration has been proposed to strengthen training integrity. Each round of updates is recorded as a transaction on a private blockchain, creating a tamper-proof history and preventing secret modifications [29].

All these protections come with trade-offs: more security typically means slower training or lower model accuracy due to added noise, encryption overhead, or verification steps [1].

5.7 Algorithm Selection Criteria

Choosing the right algorithm for a given federated deployment depends on several characteristics of the system.

Data heterogeneity. When client distributions differ significantly, as with buses on different routes, algorithms correcting for client drift are preferred. SCAFFOLD provides the strongest theoretical correction, while FedProx offers a simpler alternative. FedAvg works in practice but may converge slowly.

System heterogeneity. When devices have very different hardware or network speeds, FedProx handles partial participants gracefully, and FedNova ensures fair aggregation when clients take different numbers of local steps.

Asynchronous operation. In bus fleets, vehicles frequently disconnect. DAAFL and FedSA are specifically designed for this scenario. DAAFL additionally corrects for data disparity between high-mileage and low-mileage buses, making it the most suitable choice for realistic fleet deployments.

Communication budget. FedAvg with many local epochs minimizes communication rounds but increases drift risk. FedSGD minimizes local compute but maximizes rounds and is only suitable when bandwidth is unconstrained.

Convergence considerations. For synchronous FL with well-behaved objectives, convergence rate depends on the number of clients, local steps, and learning rate [25]. Non-IID data slows convergence because each client’s update reflects its own local distribution. In asynchronous settings, stale updates introduce an additional error: a client trained on a model that is s rounds old may submit an update misaligned with the current global state. FedSA and DAAFL both limit this staleness effect. In practice, for bus fleet predictive maintenance, **DAAFL is the most suitable algorithm**, followed by FedSA when data disparity is less of a concern.

6 Federated Learning for Predictive Maintenance in Bus Fleets

6.1 Context and Motivation

Predictive maintenance consists of using sensor data collected from vehicles to anticipate equipment failures before they occur, rather than waiting for a breakdown or relying on fixed maintenance schedules [7]. In the context of bus fleets, this means continuously monitoring onboard systems such as the engine, brakes, cooling circuit, battery, and transmission, and training models that can detect early warning signs of degradation.

Applying machine learning to bus maintenance is highly valuable: unplanned breakdowns cause service disruptions, increase repair costs, and reduce passenger trust. The centralized approach to training such models, however, presents serious obstacles. Bus sensor data contains sensitive operational information subject to data protection regulations. Collecting raw data from every vehicle to a central server is expensive in terms of bandwidth and raises privacy concerns. Federated learning directly addresses all of these constraints: sensor data remains onboard, regulations are respected, each vehicle contributes knowledge from its own operating conditions, and the system scales to fleets of any size [6].

6.2 Challenges Specific to Bus Fleets

Applying federated learning to bus fleet maintenance introduces several challenges beyond the standard FL setting.

Data heterogeneity. Each bus follows a different route, serves different passengers, operates under different traffic conditions, and may have different mechanical wear histories. The sensor data produced by each bus follows its own statistical distribution. This non-IID nature makes it harder to learn a global model that generalizes well across the fleet, and algorithms that do not account for client drift may converge to a biased solution [6].

Temporal unavailability. Buses are not always connected to the network. A vehicle may travel through tunnels, low-coverage rural areas, or be parked overnight. Standard synchronous algorithms that wait for all selected clients are poorly suited to this scenario, as a single disconnected bus would delay the entire round [6].

Data disparity. Buses on high-frequency urban routes accumulate far more sensor data than those on occasional suburban lines. Without correcting for this imbalance, the global model will be dominated by the most active vehicles and perform poorly for infrequent contributors [6].

Resource constraints. Onboard computing units vary significantly between bus models and manufacturers. Older vehicles may have limited processing power and memory, which constrains the complexity of the local model. Communication bandwidth over mobile networks is also variable and costly [1].

6.3 Federated Learning Framework for Bus Predictive Maintenance

The general framework for applying FL to bus predictive maintenance follows the lifecycle described in Section 5.4, adapted to the fleet’s constraints. Each bus continuously collects sensor readings from its onboard systems, including engine temperature, coolant level, battery state of charge, brake pad thickness, GPS speed, and fuel consumption. These readings stay entirely on the bus. A local model is trained on this local dataset and only the trained model weights are sent to the aggregation server, not the raw sensor data. The server combines the received updates and distributes the improved global model back to the fleet [7].

In practice, the aggregation can be organized hierarchically: buses within the same depot or operating on the same corridor first aggregate their updates locally through a depot server or an RSU, and the depot-level models are then forwarded to the central cloud server for global aggregation. This reduces the communication load on the central server and takes advantage of the geographic clustering of buses with similar operating conditions [9].

6.4 Models Used for Local Training

The choice of local model architecture directly impacts prediction quality and feasibility of onboard deployment.

LSTM (Long Short-Term Memory). LSTM networks capture long-range temporal dependencies in sequential sensor data [7]. An LSTM trained on several weeks of engine temperature readings can learn gradual degradation patterns invisible to models looking at individual time steps. FL-LSTM with 8 aggregation rounds has been shown to improve prediction quality by 19% compared to a standalone LSTM trained on a single vehicle [7]. The main drawback is computational cost, which may exceed the capacity of resource-constrained onboard units.

GRU (Gated Recurrent Unit). GRU is a simplified variant of LSTM with fewer parameters, which makes it faster to train and cheaper to transmit as model updates [7]. It achieves comparable accuracy on short-to-medium time-series and is better suited for buses with limited onboard resources. FL-GRU is preferred when communication cost and device constraints are the primary concerns.

FL with XAI (Explainable AI). A recent direction combines federated learning with explainability techniques such as SHAP values or attention mechanisms. This allows maintenance crews to understand not just what the model predicts, but why a particular fault is flagged. Studies have achieved over 98% accuracy while maintaining full interpretability, which is critical for operational trust [11].

6.5 Aggregation Strategy

The standard weighted average (FedAvg aggregation) weights each client’s contribution proportionally to its dataset size:

$$w_{\text{global}} = \sum_{k=1}^K \frac{n_k}{n} w_k \quad (11)$$

However, dataset size alone is not always the best criterion. Performance-based weighting, which assigns higher influence to clients whose local model achieves lower validation loss, has been shown to produce better global models in bus fleet settings [7]:

$$\alpha_k = \frac{1/\text{MSE}_k}{\sum_{j=1}^K 1/\text{MSE}_j}, \quad w_{\text{global}} = \sum_{k=1}^K \alpha_k w_k \quad (12)$$

6.6 Key Results from Transportation FL Studies

Weighting client contributions based on data volume and update frequency improves global model performance [6]. Asynchronous training with DAAFL maintains good prediction quality even when vehicles disconnect for 15 to 75 seconds [6]. Federated learning reduces communication costs by over 23% compared to centralized approaches [10]. Combining FL with explainability achieves over 98% accuracy while remaining interpretable [11]. Hierarchical architectures with roadside units as intermediate aggregators improve scalability for city-scale deployments [9]. Multi-round averaging with 8 rounds achieves 19% better failure prediction than a standalone LSTM [7]. Finally, pruning and quantization techniques should be applied to keep models small and reduce per-round communication costs [1].

References

- [1] Ibrahim Abdul Majeed et al., “Comparative assessment of federated and centralized machine learning,” *arXiv:2202.01529*, 2022.
- [2] Georgios Drainakis et al., “Federated vs. centralized machine learning under privacy-elastic users,” *IEEE NCA*, pp. 1–8, 2020.
- [3] Brendan McMahan et al., “Communication-efficient learning of deep networks from decentralized data,” *AISTATS*, pp. 1273–1282, PMLR, 2017.
- [4] Yang Liu et al., “Federated learning-powered visual object detection for safety monitoring,” *DOI: 10.1609/aaai.12002*, 2021.
- [5] Glory Nosawaru Edege and Samuel Acheme, “A systematic review of centralized and decentralized machine learning models,” *NIPES Journal*, vol. 6, no. 4, pp. 161–175, 2024.
- [6] Leonie von Wahl et al., “Data disparity and temporal unavailability aware asynchronous federated learning for predictive maintenance on transportation fleets,” Volkswagen Group Technical Report, 2023.
- [7] Bohdan Shubyn et al., “Federated learning for improved prediction of failures in autonomous guided vehicles,” *Journal of Computational Science*, 2023.
- [8] Konstantinos Lazaros et al., “Federated learning: Navigating the landscape of collaborative intelligence,” *Electronics*, vol. 13, p. 4744, 2024.
- [9] Mario S. Vela Romo et al., “Federated learning frameworks for intelligent transportation systems,” *MDPI*, 2025.
- [10] Tariq Alqubaysi et al., “Federated learning-based predictive traffic management,” *MDPI Sensors*, vol. 25, p. 1116, 2025.
- [11] Hamad M. H. A. Alzari et al., “Privacy-preserving interpretability: An explainable federated learning model,” *MDPI*, 2025.
- [12] Muhammad Asad, Ahmed Moustafa, and Takayuki Ito, “Federated learning versus classical machine learning,” *arXiv:2107.10976*, 2021.
- [13] Dana Alsagheer, Lei Xu, and Weidong Shi, “Decentralized machine learning governance,” *IEEE Access*, 2023.
- [14] Georgios A. Kaissis et al., “Secure, privacy-preserving and federated machine learning in medical imaging,” *Nature Machine Intelligence*, vol. 2, no. 6, pp. 305–311, 2020.
- [15] Darshan Naik and Nikita Naik, “The changing landscape of machine learning,” *UK Workshop on Computational Intelligence*, pp. 18–28, Springer, 2023.
- [16] Jonas Geiping et al., “Inverting gradients – how easy is it to break privacy in federated learning?” *arXiv:2003.14053*, 2020.
- [17] Cynthia Dwork and Aaron Roth et al., “The algorithmic foundations of differential privacy,” *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3–4, pp. 211–407, 2014.
- [18] Keith Bonawitz et al., “Practical secure aggregation for privacy-preserving machine learning,” *ACM CCS*, pp. 1175–1191, 2017.
- [19] Stephen Hardy et al., “Private federated learning on vertically partitioned data,” *arXiv:1711.10677*, 2017.
- [20] Tian Li et al., “Federated learning: Challenges, methods, and future directions,” *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, 2020.
- [21] Mingzhe Chen, Boming Mao, and Ting Ma, “FedSA: A staleness-aware asynchronous federated learning algorithm,” *Future Gen. Comput. Syst.*, vol. 120, pp. 1–12, 2021.

- [22] Sai Praneeth Karimireddy et al., “SCAFFOLD: Stochastic controlled averaging for federated learning,” *ICML*, pp. 5132–5143, PMLR, 2020.
- [23] Jianyu Wang et al., “Tackling the objective inconsistency problem in heterogeneous federated optimization,” *NeurIPS*, vol. 33, pp. 7611–7623, 2020.
- [24] A. Bény, E. Kordík, J. Kratochvíl, and I. Zolotárová, “Federated learning for edge computing: A survey,” *Applied Sciences*, vol. 12, no. 18, p. 9124, 2022.
- [25] Sebastian U. Stich, “Local SGD converges fast and communicates little,” *arXiv:1805.09767*, 2018.
- [26] Mario S. Vela Romo et al., “Federated learning frameworks for intelligent transportation systems,” *Smart Cities*, vol. 9, no. 12, 2026.
- [27] Joahannes R. F. Souza, Stefano Z. L. N. Oliveira, and Helio Oliveira, “The impact of federated learning on urban computing,” *Journal of Internet Services and Applications*, vol. 15, no. 1, 2024.
- [28] Vishnu Pandi Chellapandi et al., “Federated learning for connected and automated vehicles,” *IEEE Trans. Intelligent Vehicles*, early access, 2023.
- [29] Sergey Trofimov, Leonid Voskov, and Mikhail Komarov, “Decentralized public transport management system based on blockchain technology,” *Applied Sciences*, vol. 15, no. 3, p. 1348, 2025.
- [30] Yanbin Zhao, “Robust federated learning approach for travel mode identification from non-IID GPS trajectories,” 2022.